

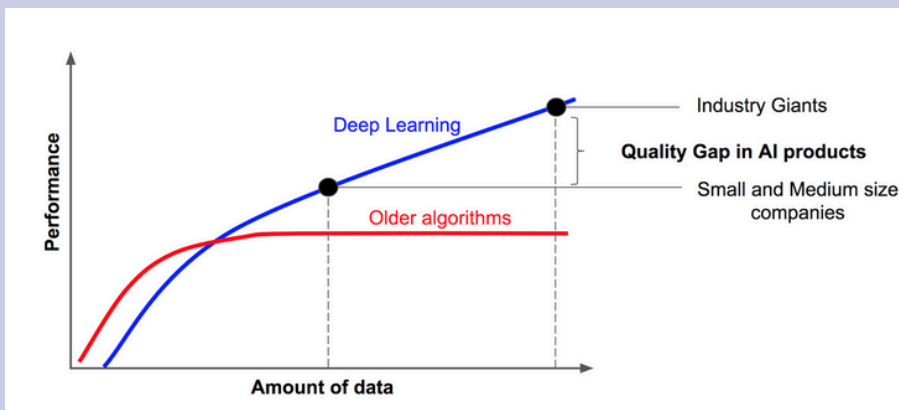
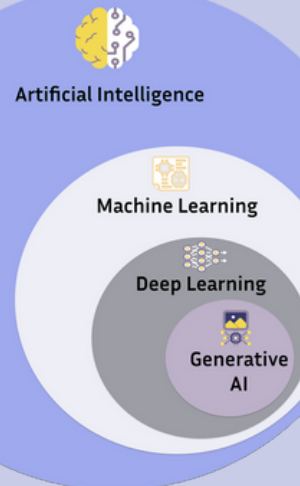
DEEP LEARNING

Prepared by: Sahab AL-Amri



what is deep learning

Deep learning is a subset of machine learning that uses multilayered neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain.



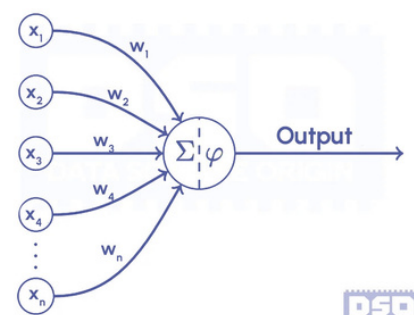
Neural $\Rightarrow$ Fundamental unit of the nervous system

Network $\Rightarrow$ Connected elements

They are the fundamental units of artificial neural networks that:

- Receive inputs
- Transform information
- Create an output

The structure of an artificial neuron.





What is a Neuron?

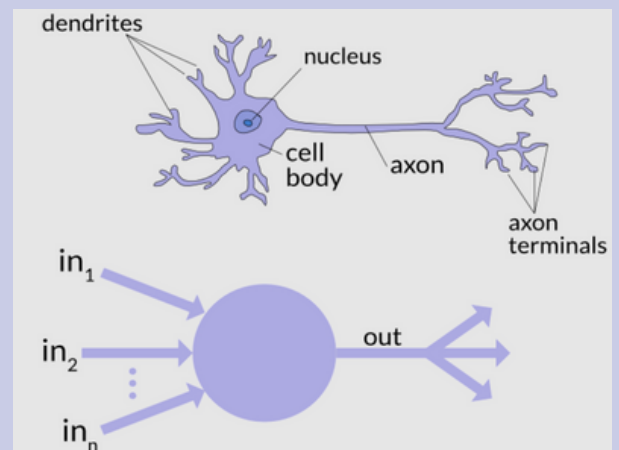
is the basic unit of a neural network. It receives input values, processes them using weights and a bias, and produces an output through an activation function. Neurons enable machines to learn patterns and make decisions from data.



What is Neural Network?

are machine learning models that mimic the complex functions of the human brain.

These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.



Layers in Neural Network Architecture

Input Layer:

This is the first layer. It receives the data given to the network. Each neuron represents one piece of the input data.

Hidden Layers:

These are the middle layers. They do most of the thinking and learning by finding patterns in the data.

Output Layer:

This is the last layer. It gives the final result or prediction, depending on the task (such as classification or prediction).



Applications of Neural Networks

1. Image and Video Recognition

- Face Unlock:
- Autonomous Driving
- Medical Imaging

2. Natural Language Processing (NLP)

- ChatGPT / Voice Assistants
- Translation
- Sentiment Analysis

3. Healthcare

- Disease Prediction
- Drug Discovery
- Personalized Treatment



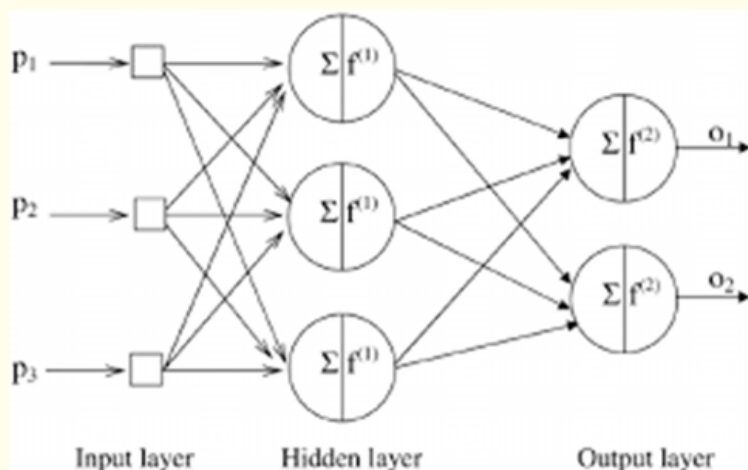
Activation Functions in NN

What is Activation Function?

An activation function determines how much information a neuron passes forward by converting its input into an output signal. It is essential because it adds non-linearity to the neural network, enabling it to learn complex, real-world patterns and control when a neuron should be activated.

Why Activation Function?

- An activation function introduces non-linearity into the model, enabling it to learn complex patterns beyond simple linear relationships.
- Without activation functions, a neural network would behave like a basic linear regression model.



Types of Activation Functions

ReLU (Rectified Linear Unit)

Used in hidden layers to introduce non-linearity while keeping computation efficient.

Sigmoid

Used in binary classification problems

Softmax

Used for multi-class classification

Tanh

Similar to sigmoid but with better gradient properties

When to Use Which Activation Function

ReLU (Rectified Linear Unit) Used in hidden layers

- Fast and computationally efficient
- Helps deep networks learn complex patterns
- Reduces vanishing gradient problem

Sigmoid Used in binary classification output layer

- Outputs values between 0 and 1
- Interpretable as probability

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Softmax Used in multi-class classification output layer

- Converts outputs into class probabilities
- All probabilities sum to 1

Tanh Used in hidden layers (sometimes)

- Outputs values between -1 and 1
- Better gradient flow than sigmoid

* Vanishing Gradients

If gradients become very small, the weights of earlier layers:

- Update extremely slowly
- Or stop learning completely

Why does it happen?

1. Activation functions like Sigmoid or Tanh

- Their derivatives are less than 1
- Repeated multiplication through many layers $\rightarrow$ gradients shrink

2. Deep networks

- More layers = more multiplications during backpropagation

3. Poor weight initialization

- Very small weights make gradients smaller layer by layer

* Back Propagation

Backpropagation is an algorithm used to train neural networks

It works by:

1. Calculating the error at the output

2. Sending this error backward

3. Updating weights to reduce the error

2. Forward pass (make a prediction)

What happens:

- Each neuron receives inputs
- Multiplies them by weights
- Adds a bias
- Applies an activation function
- Passes the result forward

This is how the network makes a guess based on current knowledge (weights).

3. Compare prediction with the correct answer

What happens:

- The network output is compared with the true target value
- The difference is called the error

The network must know how wrong it is before it can improve.

4. Measure how bad the error is

All output errors are combined into a single number (total error).

This gives one clear signal:

Large error → bad prediction

Small error → good prediction

The goal of learning is to reduce this error.

5. Start from the output layer

We begin correcting weights connected to the output neurons.

Why:

- Output neurons directly cause the error
- They are the easiest place to assign blame

Think of it as:

“The mistake happened here first.”

6. Decide how responsible each weight is

The network evaluates each weight to see how much it influenced the output and whether changing it increases or decreases the error. Weights that contribute more to the error are adjusted more, while less influential weights are changed slightly.

7. Adjust output weights slightly

Important weights are adjusted more than less important ones, and all updates are kept small. These small steps prevent overcorrection and instability, allowing the network to learn gradually rather than making sudden changes.

8: Send error backward to the hidden layer

Hidden neurons receive feedback from output neurons indicating how much they contributed to the error. Since hidden neurons do not see the final error directly, they learn through the neurons they connect to—this process is known as backpropagation.

9: Adjust hidden layer weights

What happens:

Weights between input and hidden layer are updated.

Why:

Hidden neurons shape the features passed forward.

If they improve, the output will also improve.

10: Update biases

What happens:

Bias values are adjusted just like weights.

Why:

Bias controls when a neuron activates.

Adjusting it helps the neuron respond better to inputs.

One learning cycle is complete

The network makes a prediction, measures the error, and updates itself—completing one training iteration. These steps are repeated many times across many examples, and with each cycle, the network improves gradually.

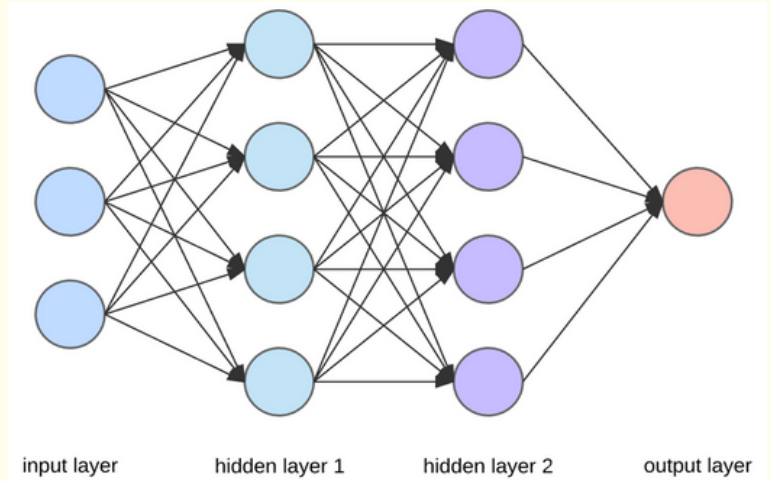


What is dense layer

In TensorFlow, `tf.keras.layers.Dense` represents a fully connected layer where each neuron is connected to all neurons in the previous layer. It is used to learn complex patterns and relationships in data.

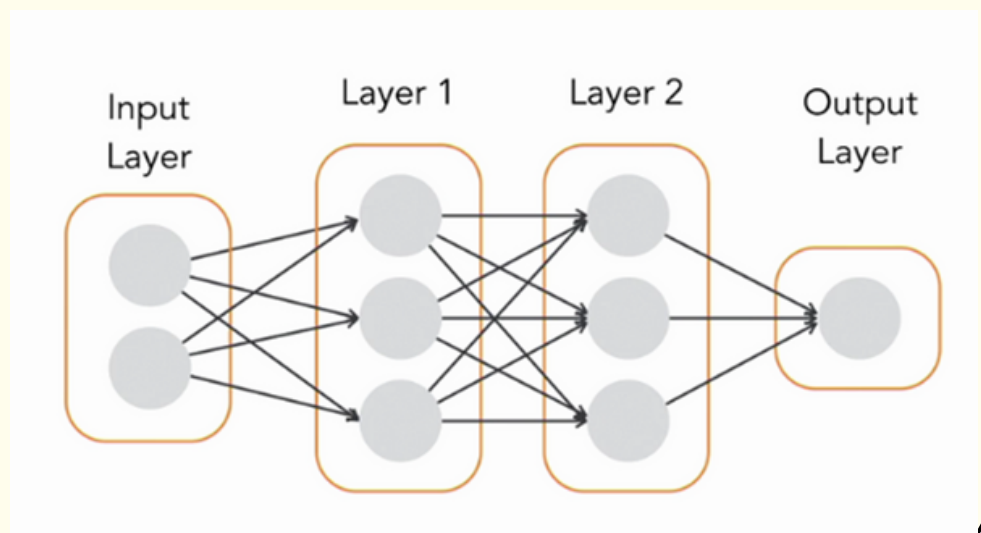
Mathematical operation:

$\text{output} = \text{activation}(\text{dot}(\text{input}, \text{kernel}) + \text{bias})$



Using Dense Layer in a Sequential Model

The Sequential model is a linear stack of layers where each layer has exactly one input and one output. It is ideal for building simple neural networks in which layers are added one after another, such as basic feedforward or convolutional models with a single input-output path.





Using Dense Layer with Sequential Model

The Sequential model is used for simple neural networks with layers stacked linearly, where each layer has one input and one output.

It is suitable for basic models like feedforward networks.

Not recommended when:

- The model has multiple inputs or outputs
- Any layer has multiple inputs or outputs
- Layer sharing is needed
- A non-linear topology is required



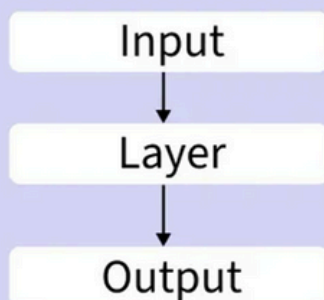
Using Dense Layer in the Functional API

The Functional API is more flexible and powerful, allowing the creation of complex architectures with:

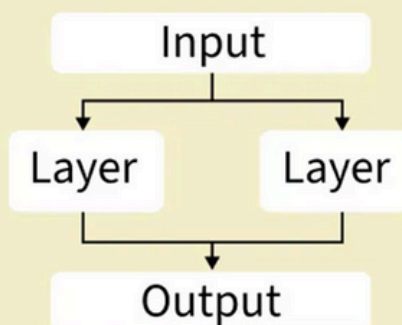
- Multiple inputs and outputs
- Shared layers
- Non-linear connections such as branching and skip connections

It is ideal for advanced neural networks that cannot be represented by a simple linear stack of layers.

Sequential API



Functional API





Convolutional Neural Networks (CNNs)

How a computer sees images?

Computers do not see like humans; they see numbers.

Images are made of pixels, each with numerical values:

Black - 0

White - 255

Color images - 3 numbers per pixel (RGB)

Humans see meaning → Computers see numbers → detect patterns → understand meaning

Computer Vision Tasks

Image Classification - Identify what is in the image

Object Detection - Locate objects and identify them (e.g., car here, person here)

Image Segmentation - Label every pixel in the image (e.g., background / road / car)

Example Model: YOLO - detects objects in images/videos in real time

Why CNNs?

MLPs fail at images because they ignore spatial structure

CNNs capture local patterns using filters



Key CNN Concepts

Sliding Window

- Move a small filter (kernel) over the image
- Detects local features (edges, textures)
- Same filter is applied across all positions → reduces number of weights

Feature Map

- Output of a convolutional layer
- Stores patterns detected by filters in a 2D grid

Flatten Layer

- Converts multi-dimensional data into a 1D vector
- Does not change values, only shape

Pooling

- Reduces feature map size while keeping important info
- Types:
 - Max Pooling - takes maximum value in a small window
 - Average Pooling - takes average value
- Stride - 2 → halves width & height

Benefits of Pooling

- Reduces computation and memory usage
- Provides translation invariance
- Highlights important features
- Helps prevent overfitting

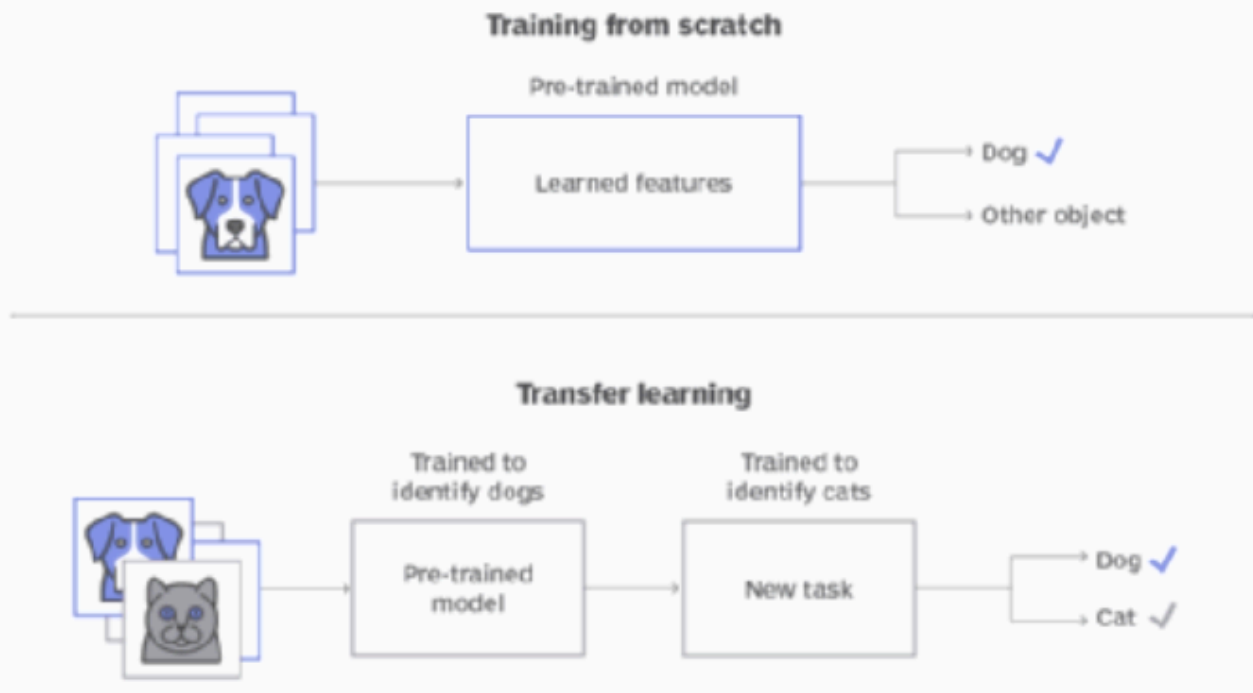
Padding

- Add extra pixels around image borders before convolution
- Keeps output size same as input
- Preserves edge information

* Transfer Learning

Transfer Learning is a deep learning technique where a model trained on a large, general dataset is reused and adapted for a new but related task, instead of training a model from scratch.

How transfer learning works



Core Idea

A pretrained model has already learned useful features.

For example, in image tasks:

Early layers learn edges and textures

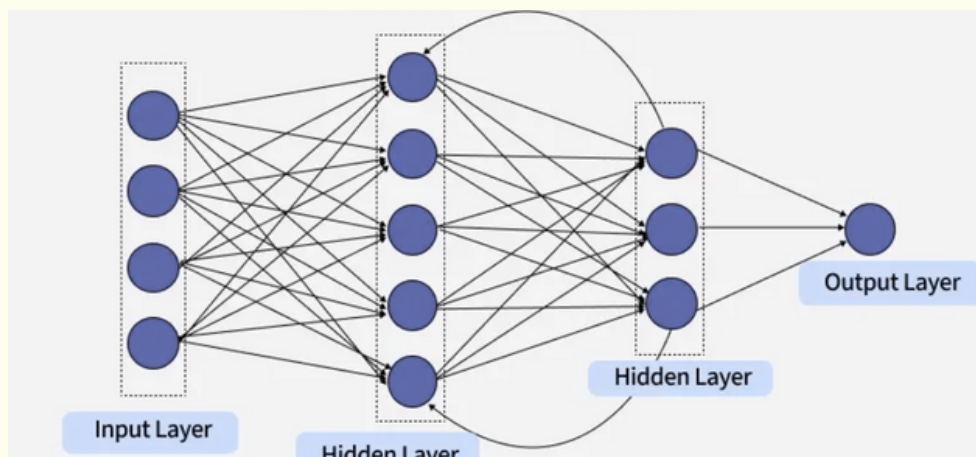
Later layers learn shapes and objects

We reuse these learned features for a new problem.

✿ RNN (Recurrent Neural Network)

Recurrent Neural Networks (RNNs) are a class of neural networks designed to work with sequential data. Unlike traditional feedforward neural networks, RNNs have internal memory that allows them to retain information from previous time steps. This makes them especially suitable for tasks where the order of data is important.

An RNN processes input data step by step, and at each step, it combines the current input with information from the previous state. This enables the network to learn temporal dependencies and patterns over time. RNNs are commonly used in applications such as text processing, speech recognition, time-series prediction, and natural language processing.



✿ Sequential Data

Sequential data refers to data where the order of elements matters, and each element is related to the previous ones. In this type of data, the current value often depends on past values.

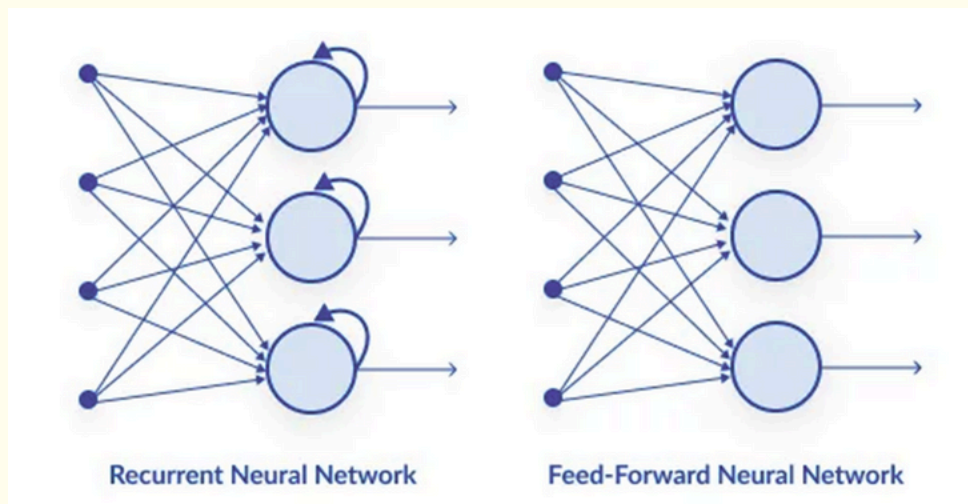
Examples of sequence data

Speech recognition		→ "The quick brown fox jumped over the lazy dog."
Music generation		→ 
Sentiment classification	"There is nothing to like in this movie."	→ ★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAAGTAG	→ AGCCCCTGTGAGGAAGTAG
Machine translation	Voulez-vous chanter avec moi?	→ Do you want to sing with me?
Video activity recognition		→ Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→ Yesterday, Harry Potter met Hermione Granger .

Because sequential data contains temporal relationships, models like RNNs, LSTMs, and GRUs are widely used to analyze and learn from this type of data.

✿ Why Feedforward NN fails?

- Feedforward neural networks process inputs in a single direction from input to output without any notion of sequence or memory.
- They assume that all inputs and outputs are independent of each other.
- This makes them unsuitable for tasks where the context or sequence of data points is crucial.
- Sequential data requires specialized models that can remember past information, and in some cases, also use future context when the entire sequence is available.
- Requires models that capture temporal dependencies



✿ What is RNNs

- Recurrent Neural Networks (RNNs) – Understanding Sequential Data
- Designed to handle sequences where order matters
- Can remember past information and, in some models, leverage future context
- They are essential for tasks that involve time series data or any data where the order matters.

✿ RNN Overview

Special type of neural network for sequential data

Key features:

Internal memory → stores past information

Contextual understanding → current input + past context

Dynamic processing → updates memory with each new input



Weather Example Analogy

- Meteorologist reading daily temperatures
- Input - today's temperature
- Hidden state - memory of previous days
- Loop - same meteorologist updates memory every day
- Output - prediction for tomorrow's weather

Weather Example Analogy

$h_{t-1} \rightarrow$ previous memory

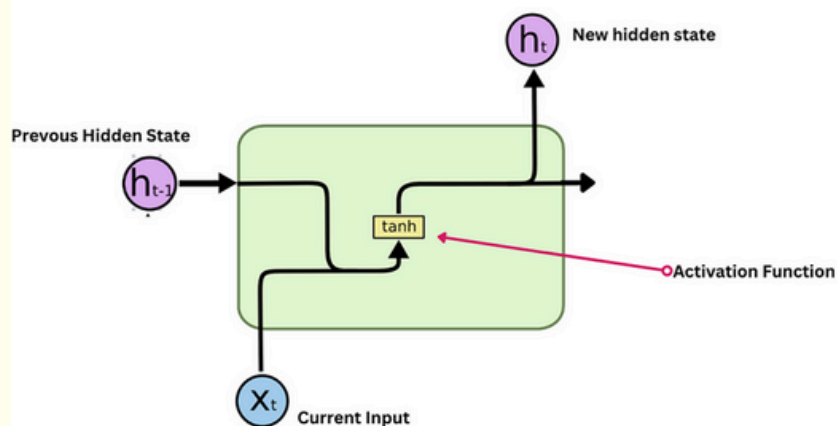
$x_t \rightarrow$ current input

$W_h \rightarrow$ weight for past memory

$W_x \rightarrow$ weight for current input

$h_t \rightarrow$ updated memory

RNN (Mathematical Intuition)



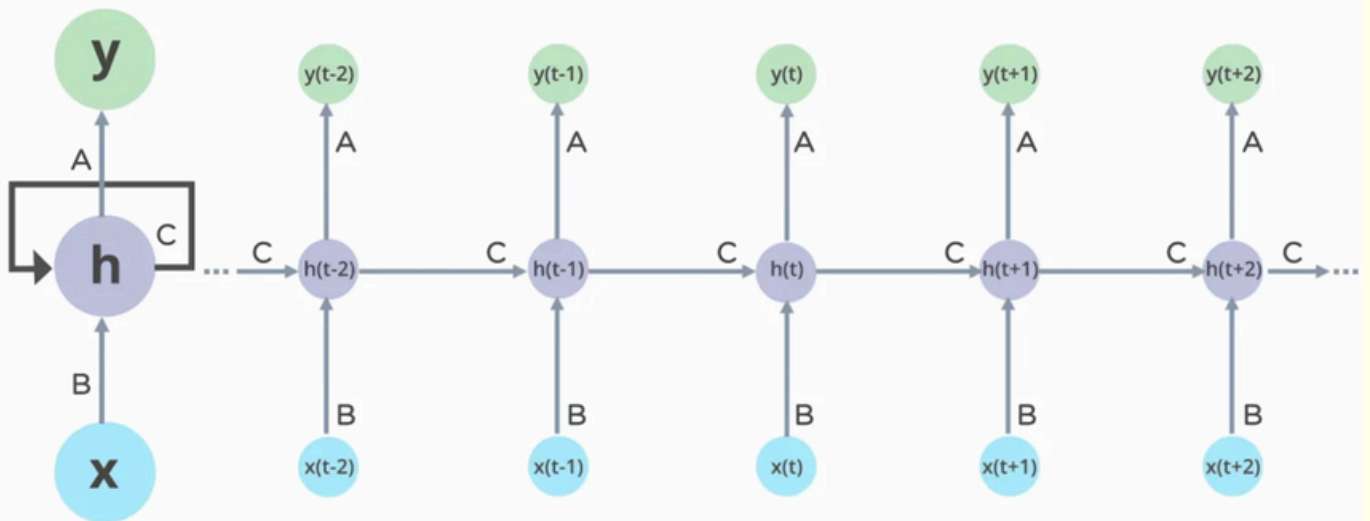
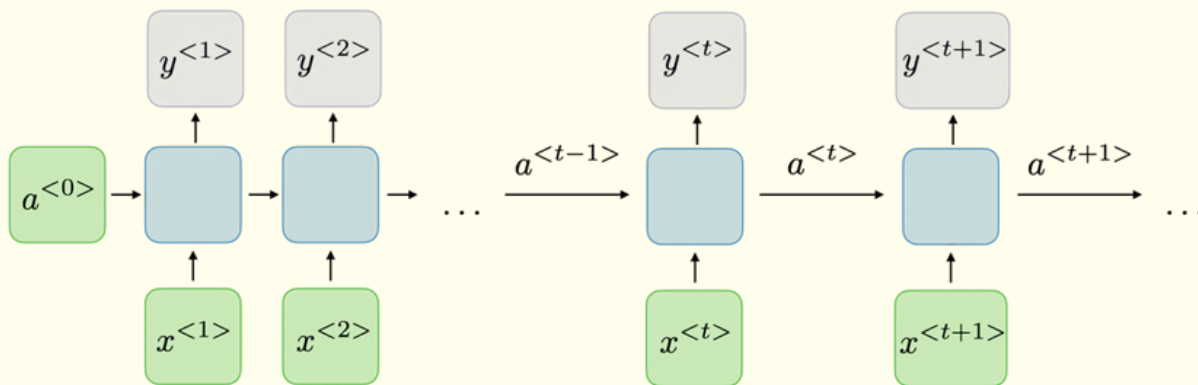
$$h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

$$h_t = fw(x_t, h_{t-1})$$

New hidden state Activation Function Previous Hidden State

✿ How RNN works?

An RNN processes a sequence of inputs, maintaining a hidden state that captures information about the sequence.



✿ Step-by-Step Weather Example

Sequence: 30, 32, 35 Temperature

Initial $h_0 = 0$, $W_x = 0.1$, $W_h = 0.8$, $b = 0$

$$h_1 = \tanh(0.1 \cdot 30 + 0.8 \cdot 0) \approx 0.995$$

$$h_2 = \tanh(0.1 \cdot 32 + 0.8 \cdot 0.995) \approx 0.999$$

$$h_3 = \tanh(0.1 \cdot 35 + 0.8 \cdot 0.999) \approx 0.9999$$

Hidden state tracks the increasing temperature trend

Day 1: Temp-30, Hidden State-0.9951, Output-0.9951

Day 2: Temp-32, Hidden State-0.9991, Output-0.9991

Day 3: Temp-35, Hidden State-0.9999, Output-0.9999

Notice how the hidden state grows slightly, reflecting the increasing trend in temperature.

* LSTM

LSTM (Long Short-Term Memory) is a special type of Recurrent Neural Network (RNN) designed to overcome the limitations of traditional RNNs. Standard RNNs struggle with long-term dependencies

LSTM can remember important information for long periods

→ LSTM is widely used for:

- Time series prediction
- Natural Language Processing (NLP)
- Speech recognition
- Machine translation

→ LSTM uses gates:

1. Forget Gate – decides what to forget
2. Input Gate – decides what to store
3. Output Gate – decides what to output

This makes LSTM powerful for sequential data.

* Generative Models in Deep Learning

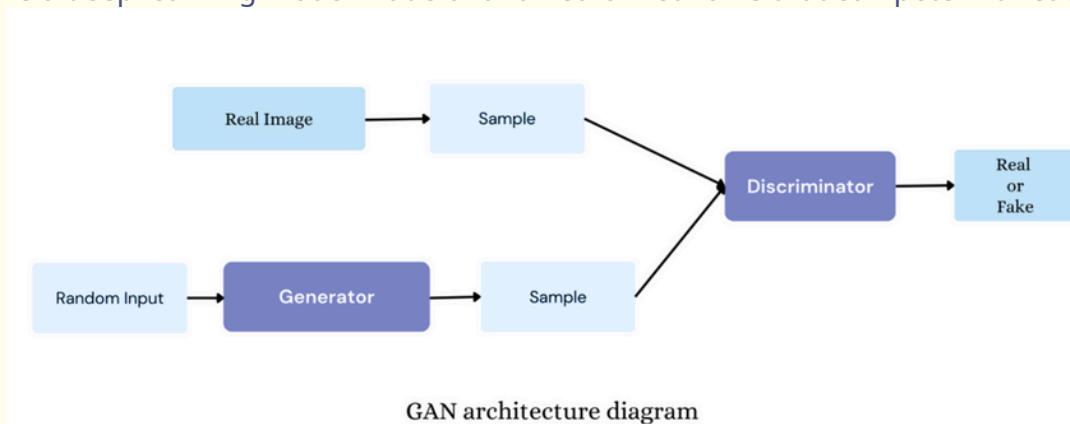
GANs and Autoencoders

Generative models are deep learning models that learn from existing data and generate new data that looks realistic.

Instead of only classifying data, these models can create images, signals, or samples. Two important generative models are GANs and Autoencoders.

I. What is a GAN (Generative Adversarial Network)?

A GAN is a deep learning model made of two neural networks that compete with each other.



GAN architecture diagram

GAN Components

Generator

Creates fake data (images, for example) from random noise
Tries to make the fake data look real

Discriminator

Acts like a judge

Decides whether the input data is real or fake

* How GAN Training Works

1. Generator creates a fake image
2. Discriminator checks:
 - Real image $\rightarrow$ label = 1
 - Fake image $\rightarrow$ label = 0
3. Discriminator gives feedback
4. Generator improves
5. Process repeats until fake data looks real

💡 **Simple analogy:**

Generator = counterfeiter

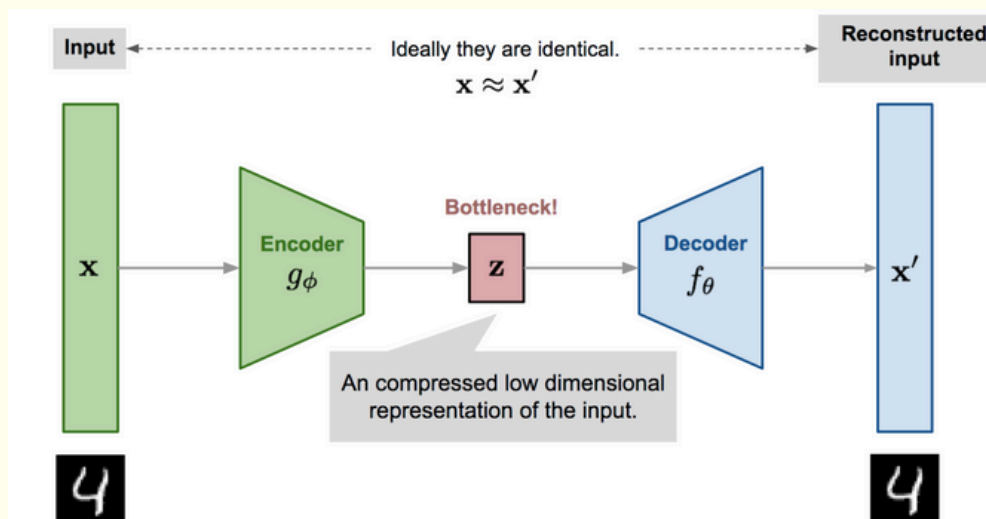
Discriminator = police officer

* Applications of GANs

1. Image generation (faces, objects)
2. Data augmentation
3. Style transfer
4. Image-to-image translation

* What is an Autoencoder?

An Autoencoder is a neural network that learns how to compress data and then reconstruct it.



* Autoencoder Components

- **Encoder**
 - Compresses the input data
 - Produces a smaller representation called Latent Space
- **Latent Space**
 - A low-dimensional space
 - Stores the most important features only
- **Decoder**
 - Reconstructs the original data from the latent space

* How Autoencoders Learn

- The model compares:
 - Original input
 - Reconstructed output
- It minimizes the reconstruction error (e.g., MSE)

* Applications of Autoencoders

- Dimensionality reduction
- Denoising images
- Feature extraction
- Anomaly detection

Feature	GANs	Autoencoders
Main Goal	Generate new realistic data	Compress and reconstruct data
Architecture	Generator + Discriminator	Encoder + Decoder
Input	Random noise	Real data
Output	New synthetic data	Reconstructed input
Training Method	Adversarial (competition)	Reconstruction loss
Stability	Difficult and unstable	Easier and stable
Data Generation	Yes (main purpose)	Limited (mainly VAEs)
Common Use	Image generation	Denoising, compression